Local Fish Species Recognition with EfficientNetB0 and Confidence-Based Rejection

Szymon Wilczek `sw312468` **Bartosz Kaprak** `bk311868`

Project Developed as Part of Academic Coursework

May 23, 2026

Abstract

This paper describes *Sonar Shark*, a course project for image-based fish species recognition. The system classifies 96 categories, including Polish freshwater and Baltic species, sharks, selected marine animals, and additional commercial or non-native freshwater species. Training data was collected with a Python image scraper using multilingual search queries and then cleaned with automated image validation. The classifier uses EfficientNetB0 with Keras augmentation layers and a small custom classification head. The trained model is used locally in a Tkinter desktop application, which can load `.keras` model files from the application directory and display species descriptions from a CSV file. The final local inference code also applies a simple confidence threshold of $\tau = 35.0\%$ to reject low-confidence inputs. This threshold is useful for some non-target images, but it is not a full open-set recognition method and can still produce false positives near the decision boundary.

Keywords: Computer Vision, EfficientNetB0, Image Classification, Web Scraping, Confidence Thresholding, Fish Recognition.

1 Introduction

Fish species recognition from photographs can support education, recreational fishing tools, and simple field documentation. In practice, identification is still usually performed manually from visible morphological traits, such as body shape, fin position, scale pattern, coloration, and the presence or absence of barbels. This is reliable when done by an expert, but it is slower and less accessible for casual users.

Convolutional neural networks are well suited to this type of image classification task, but the dataset is naturally difficult. Fish photographs often contain cluttered backgrounds, hands, nets, reflections, low resolution, motion blur, and inconsistent lighting. Several species are also visually similar. For example, roach and rudd are both cyprinids with similar body profiles and red fins, so small details such as eye color and fin alignment can matter.

This project implements *Sonar Shark*, a 96-class classifier developed for the Biologically Inspired Artificial Intelligence (BIAI) course. The class list includes local Polish freshwater and Baltic species, such as *Esox lucius*, *Perca fluviatilis*, and *Cyprinus carpio*, as well as sharks, selected marine animals, and additional globally common aquaculture or freshwater species.

The primary engineering contributions of this project are:

1. **Data collection and cleaning:** A local Python scraper downloads image candidates from search results and removes unreadable files with `fastai.vision.utils.verify_images`.
2. **Model architecture:** The classifier uses EfficientNetB0 without the original ImageNet classification top, followed by global average pooling, dropout, and a 96-unit Softmax output layer.
3. **Local application:** A Tkinter GUI loads images, selects a local `.keras` model file, runs inference, and displays the predicted class with a matching species description when available.
4. **Confidence-based rejection:** A post-processing threshold rejects predictions whose highest Softmax score is below 35.0%. This reduces some invalid outputs, while still leaving known edge cases.

The remainder of this paper is organized as follows. Section 2 describes data collection and dataset taxonomy. Section 3 covers the model structure and training workflow. Section 4 describes the local inference code and confidence threshold. Section 5 discusses observed results and limitations. Section 6 summarizes the project and future work.

2 Automated Data Acquisition and Preprocessing

Classification quality depends strongly on the images used for training. Since ready-made datasets for Polish freshwater species are limited, the project uses a custom image collection script and a manually organized directory structure.

2.1 Multi-Lingual Querying Logic

Data collection is handled by `scraper.py`. The script defines a dictionary of Polish species names and search phrases, creates a destination folder for each species, queries an image search API, downloads the returned image URLs, and validates the downloaded files.

Each species key in `gatunki_pl_total` is mapped to a query string that usually combines a Polish common name, a Latin or English species name, and sometimes an environmental keyword. For example:

- *Northern Pike:* `"szczupak ryba esox lucius underwater"`
- *Common Carp:* `"karp ryba cyprinus carpio underwater"`

- *European Perch:* `"okoń europejski perch underwater"`

This does not guarantee clean data, but it improves the chance of retrieving biological photographs instead of unrelated images such as prepared food, fishing gear, or diagrams.

2.2 Search API and Rate-Limiting

The current repository implementation uses the `DDGS` image search interface. Because image search providers may rate-limit repeated automated requests, the script retries failed searches and waits between attempts. It also sleeps for a random interval between 6 and 12 seconds after each species. This makes the scraper slower, but reduces the number of failed requests during long runs.

2.3 Automated Stream Validation and Cleaning

Automated image downloading often produces truncated files, unsupported formats, empty files, or HTML error pages saved with image extensions. The script therefore validates the downloaded files before the dataset is used for training.

The `verify_images()` function from `fastai.vision.utils` scans the downloaded images and returns files that cannot be decoded. These files are then removed with `Path.unlink`. This step is important because corrupted images would otherwise fail later during image loading or batch creation.

2.4 Dataset Taxonomy and Manual Verification

The validated images are stored in a folder named `MASTER_DATASET`, with one subfolder per class during dataset preparation. The final local model contains 96 output classes listed in `SonarApp/sonar.py`. The class prefixes divide the model scope into four groups:

1. `PL_` **(Polish and Baltic species):** 49 classes representing freshwater and Baltic Sea fish (e.g., Zander, Ide, Tench, Wels Catfish, Common Roach).
2. `Shark_`**:** 14 classes containing globally monitored shark lineages (e.g., Great White, Tiger, Hammerhead, Mako).
3. `Ocean_`**:** 8 categories encompassing marine fauna (e.g., Sea Turtle, Jellyfish, Lobster, Squid).
4. `Turbo_`**:** 25 commercial, aquaculture, and non-indigenous freshwater fish species (e.g., Tilapia, Bighead Carp, Walking Catfish).

Search queries can still return wrong images, including diagrams, prepared meals, logos, or photographs of other species. For that reason, the automatic cleaning step should be treated only as technical validation. Taxonomic correctness still required manual inspection of the folders before training.

3 Model Architecture and Hybrid Training Workflow

The model has to distinguish 96 classes, so the architecture needs enough capacity for visual features while remaining practical to load on a normal laptop. The implemented model uses EfficientNetB0 as the convolutional backbone and a small classification head.

3.1 Convolutional Neural Network Selection

The *EfficientNetB0* architecture was selected as the feature extraction backbone. EfficientNet scales network depth (d), width (w), and input resolution (r) using a compound coefficient, which provides a good accuracy-to-parameter-count trade-off compared with many manually scaled CNNs [1].

EfficientNetB0 is also a reasonable choice for a student project because it is available directly in `tf.keras.applications`, has a known input size of 224×224, and can be reconstructed reliably before loading saved weights. In the local repository, this reconstruction happens in `SonarApp/sonar.py`.

3.2 Augmentation and Classification Head Design

The model starts with a small Keras augmentation block:

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.15)
])
```

During training, this block randomly flips, rotates, and zooms the input images. The goal is not to make the classifier invariant to every possible condition, but to reduce dependence on a single pose, crop, or background pattern.

The original EfficientNetB0 classification top is removed with `include_top=False`. The project then adds:

1. **Global Average Pooling 2D:** Converts the final feature maps into a single vector without introducing a large fully connected layer.
2. **Dropout Layer (0.3):** Applies 30% dropout during training as regularization.
3. **Dense Output Layer:** Maps the features to 96 class scores and applies *Softmax*.

The local script initializes EfficientNetB0 with `weights=None`, builds the same layer structure, and then loads the trained weights from a `.keras` file. In the current repository, the GUI discovers `sonar_shark.keras` automatically because it scans the `SonarApp` directory for files ending with `.keras`.

3.3 Distributed Training Protocol

The project separates data preparation and inference from model training. The repository contains the local scraper, the local GUI, the inference wrapper, the description loader, and the exported model file. The heavy training run was performed on cloud GPU infrastructure with Kaggle.

The reported final model, the *Shark* variant, was trained for about 200 epochs with a mini-batch size of 32. The training setup used the Adam optimizer [7]. The exported weights are loaded locally for inference rather than retrained inside the desktop application. `ReduceLROnPlateau` was also used to lower the learning rate when validation loss stopped improving.

4 Local Inference Engine and Anomaly Filtering

The trained model is used locally from the `SonarApp` directory. The desktop application loads an image, resizes it to the model input size, runs Keras inference, and displays the result.

4.1 Software Architecture and Dynamic Weight Discovery

The client-side code consists of three main files: `sonar_gui.py` for the GUI, `sonar.py` for model construction and prediction, and `descriptions_loader.py` for reading species descriptions from `descriptions.csv`.

The presentation layer is built with `tkinter`. It uses Pillow to open selected image files and `Image.Resampling.LANCZOS` to resize them for display while preserving the aspect ratio. In the current implementation, analysis is triggered from the GUI event handler and runs in the main Tkinter thread.

During initialization, the GUI scans its own directory for files with the `.keras` extension and fills a dropdown menu with the results. This allows different exported model files to be selected without changing the interface code.

4.2 Mathematical Formulation of Out-of-Scope (OOS) Filtering

The Softmax output always assigns probability mass across the known classes. A standard closed-set classifier therefore still returns a best class for images outside the training scope, such as landscapes, household objects, or unrelated animals.

To reduce this behavior, `sonar.py` applies a simple threshold to the maximum Softmax score. Let $\mathbf{x}$ be an input image resized to $224 \times 224 \times 3$. The model maps it to class probabilities $\hat{\mathbf{y}} \in \mathbb{R}^K$, where $K = 96$:

$$\hat{y}_i = \sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } i = 1, \ldots, K \tag{1}$$

The predicted class index is selected with:

$$c^* = \arg\max_i (\hat{y}_i) \tag{2}$$

The local code uses a threshold $\tau = 35.0\%$. The returned label is:

$$L(\mathbf{x}) = \begin{cases} \text{CLASS\_NAMES}[c^*], & \text{if } \hat{y}_{c^*} \times 100 \geq \tau \\ \text{"Not recognized (Object out of knowledge scope)"}, & \text{if } \hat{y}_{c^*} \times 100 < \tau \end{cases} \tag{3}$$

This should be interpreted as a heuristic, not as a mathematically complete out-of-distribution detector. In a local check with the exported model, the landscape image `lowquai.jpg` was rejected with a maximum confidence of 12.50%. The non-fish image `doniczka-kot.jpg` reached 33.63% confidence for `PL_Kielb`; with the 35.0% threshold it is also rejected. This shows why the threshold was set conservatively above this observed borderline case, while still not guaranteeing correct rejection for every out-of-scope image.

Figure 1: Example non-target image from the local examples directory. The exported model rejected this image because its top Softmax confidence was below $\tau = 35.0\%$.

4.3 Contextual Knowledge Retrieval Layer

When an image yields an in-scope classification ($\geq \tau$), the GUI looks up the returned class name in a dictionary produced by `descriptions_loader.py`. This module uses `pandas` to read the local file `descriptions.csv`.

The loader builds a Python dictionary from model class tokens to display text:

```
# Formatted metadata concatenation returned to the GUI
full_text = f"Name: {common_name}\nDescription: {desc}\nHabitat: {habitat}\nAverage
    Size: {size}"
```

This step does not influence the model prediction. It only adds readable species information to the GUI when a class name is found in the CSV file.

5 Experimental Results and Discussion

This section summarizes the reported validation result and the behavior observed in local inference examples. The emphasis is on practical limitations rather than claiming that the classifier is robust in all field conditions.

5.1 Quantitative Classification Evaluation

The final training run reached validation accuracy above **93.0%** after approximately 200 epochs. This is a strong result for a 96-class classifier trained from a custom web-scraped dataset, but it should be interpreted together with the dataset construction process. If near-duplicate images, uneven class sizes, or search-engine bias remain in the dataset, validation accuracy can overestimate performance on new photographs.

Local smoke tests with the exported `sonar_shark.keras` file show that the model loads correctly and produces plausible predictions for several example images: `PL_Ploc` for `ploc.jpg`, `PL_Karp` for `karp.jpg`, `PL_Klen` for `klen.jpg`, `Shark_hammerhead` for `rekin-mlot.jpg`, and `Ocean_Sea_turtle` for `turtle.jpg`.

5.2 Qualitative Analysis of Morphological Edge Cases

A realistic source of errors is similarity between related species. Two groups are especially important in this dataset:

- **Roach and rudd:** `PL_Ploc` and `PL_Wzdrega` have similar body shape and fin coloration. Distinguishing them often depends on details such as eye color and fin position, which may be unclear in low-quality images.
- **Carp-like species:** `PL_Karp`, `Turbo_Carp`, `PL_Amur_Bialy`, and `PL_Karas_Pospolity` are visually close enough that poor crops or low resolution can hide important features such as barbels or scale pattern.

5.3 Confidence Thresholding Trade-offs and UI Profiles

The threshold $\tau = 35.0\%$ creates a simple precision-recall trade-off. Raising the threshold rejects more uncertain predictions, but it can also reject valid fish photographs. Lowering it accepts more fish images, but increases the risk of false positives on unrelated inputs.

The local example set illustrates both sides of this behavior. A clear fish image such as `ploc.jpg` is classified with high confidence, while the landscape image `lowquai.jpg` is rejected. The borderline non-fish image `doniczka-kot.jpg`, which previously scored 33.63%, is also rejected after raising the threshold to 35.0%. This makes the filter more conservative, but it should still not be described as a reliable open-world anomaly detector.

From a software engineering perspective, the safer user experience is to avoid presenting low-confidence predictions as facts. In the current GUI, rejected inputs are displayed as `Not recognized (Object out of knowledge scope)` together with the confidence value.

Two local examples are shown below:

- **Accepted fish example:** Figure 2 shows `ploc.jpg`, which the local model predicted as `PL_Ploc` with 85.60% confidence.
- **Rejected non-fish example:** Figure 1 shows `lowquai.jpg`, which was rejected with 12.50% confidence.

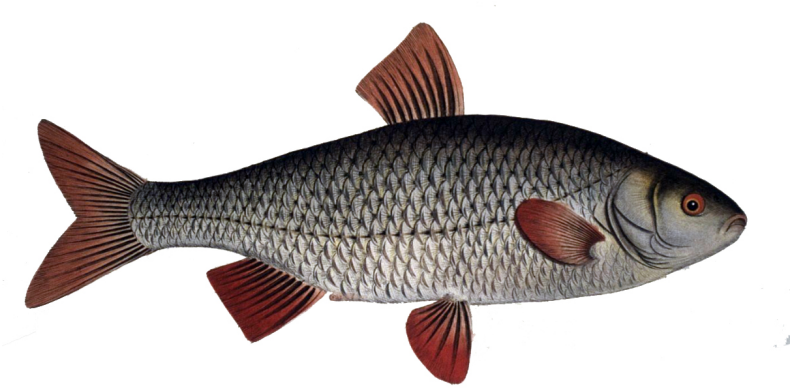

Figure 2: Accepted in-scope example from the local examples directory. The exported model predicted `PL_Ploc` with 85.60% confidence.

Figure 3: Borderline non-fish example. The model's strongest class for this image was `PL_Kielb` with 33.63% confidence, so the 35.0% threshold rejects it.

6 Conclusion and Future Work

This paper described *Sonar Shark*, a local fish and marine-species image classifier built as a BIAI coursework project. The current implementation includes a scraper, a 96-class EfficientNetB0-based classifier, a Tkinter GUI, dynamic `.keras` model loading, CSV-based species descriptions, and a confidence threshold for low-confidence predictions.

The project shows that a usable image classification prototype can be built with standard Python tools and cloud training. The validation accuracy above **93.0%** is promising, but the deployed behavior still depends on dataset quality, class balance, image conditions, and the limits of closed-set Softmax classification.

The confidence threshold is a practical addition, but it should be presented honestly: it rejects some non-target images and fails on others. For a production application, a stronger solution would require better calibration, a separate unknown-class strategy, or an explicit out-of-distribution detection method.

Future work should first improve evaluation: fixed train/validation/test splits, per-class precision and recall, a confusion matrix, and a separate test set of non-fish images would make the results easier to trust. The GUI could also move inference to a worker thread so that large models do not block the interface.

For deployment on phones, the `.keras` model could be converted to TensorFlow Lite or another mobile-friendly format. A later version could also add an object detector, such as YOLO [8], before classification, so that the classifier receives a crop around the fish instead of the whole photograph.

References

[1] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Long Beach, CA, USA, 2019, pp. 6105–6114.

[2] F. Chollet et al., "Keras," https://github.com/keras-team/keras, 2015.

[3] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," arXiv preprint arXiv:1603.04467, 2016.

[4] J. Howard and S. Gugger, "Fastai: A Layered API for Deep Learning," *Information*, vol. 11, no. 2, p. 108, 2020.

[5] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference (SciPy)*, 2010, pp. 51–56.

[6] A. Clark and Contributors, "Pillow (PIL Fork)," GitHub repository, https://github.com/python-pillow/Pillow, accessed May 23, 2026.

[7] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.